

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

**Тема: РАЗРАБОТКА ВИЗУАЛИЗАТОРА АЛГОРИТМА ТОПОЛОГИ-
ЧЕСКОЙ СОРТИРОВКИ
НА ЯЗЫКЕ JAVA**

Студент гр. 4383

Д.К. Серженко

Студент гр. 4383

А.А. Беспалов

Студент гр. 4381

В.А. Мазеев

Руководитель

Р.П. Шестопалов

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Д.К. Серженко Группа: 4383

Студент: А.А. Беспалов Группа: 4383

Студент: В.А. Мазеев Группа: 4381

Тема практики: Разработка визуализатора алгоритма топологической сортировки на языке Java

Задание на практику:

Командная итеративная разработка визуализатора алгоритма на языке Java с графическим интерфейсом. Алгоритм: топологическая сортировка ориентированного графа (алгоритм Кана).

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент	Д.К. Серженко
---------	---------------

Студент	А.А. Беспалов
---------	---------------

Студент	В.А. Мазеев
---------	-------------

Руководитель	Р.П. Шестопапов
--------------	-----------------

АННОТАЦИЯ

Отчёт содержит результаты учебной практики «Новые языки программирования». В ходе практики изучен язык Java и выполнено вводное задание, перевод лабораторной работы по дисциплине «Построение и анализ алгоритмов» (поиск расстояния Левенштейна между двумя строками) с языка Python на Java. Составлена и согласована спецификация приложения. Далее в составе бригады из трёх студентов итеративно разработан визуализатор топологической сортировки ориентированного графа (алгоритм Кана), приложение с графическим интерфейсом, пошаговой визуализацией, текстовыми пояснениями, редактированием графа мышью, загрузкой графа из файла и сохранением результата. Приложение написано на Java с использованием библиотеки Swing и собрано в исполнимый jar-файл.

SUMMARY

The report presents the results of the educational practice on new programming languages. During the practice the Java language was studied and the introductory task was completed: a laboratory work (finding the Levenshtein distance between two strings) was ported from Python to Java. A specification was agreed upon, and then a team of three students iteratively developed a visualizer of topological sorting of a directed graph (Kahn's algorithm), a GUI application with step-by-step visualization, textual explanations, mouse graph editing, loading a graph from a file and saving the result. The application is written in Java with the Swing toolkit and packaged as an executable jar.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Предметная область.....	6
1.2 Задачи практики.....	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	7
2.1 Вводное задание: перевод лабораторной работы на язык Java.....	7
2.2 Составление спецификации программы и плана разработки.....	7
2.3 Командная итеративная разработка приложения.....	10
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ.....	15
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	16
ЗАКЛЮЧЕНИЕ.....	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	18
ПРИЛОЖЕНИЕ А.....	19

ВВЕДЕНИЕ

Предметная область практики — разработка программного обеспечения на новом для студентов языке программирования Java, а также алгоритмы на графах: топологическая сортировка ориентированного графа.

Цель практики — освоить язык Java и технологию командной итеративной разработки приложений с графическим интерфейсом на примере создания визуализатора классического алгоритма.

Задачи практики: выполнить вводное задание (перевести лабораторную работу со структурой данных на динамической памяти на язык Java); составить и согласовать спецификацию и план разработки; в составе бригады итеративно разработать визуализатор топологической сортировки в соответствии со спецификацией.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Топологическая сортировка — это расположение вершин ориентированного графа в такой последовательности, что каждое ребро ведёт от более ранней вершины к более поздней. Задача возникает всюду, где есть зависимости: планирование работ, порядок сборки модулей программы, порядок изучения тем курса. Порядок существует тогда и только тогда, когда в графе нет циклов. Алгоритм Кана строит порядок так: считаются входящие степени вершин, вершины с нулевой степенью помещаются в очередь готовых; далее вершины по одной берутся из очереди, дописываются в порядок, их исходящие рёбра удаляются, а соседи с обнулившейся степенью добавляются в очередь [1]. Если очередь опустела раньше, чем кончились вершины, значит в графе есть цикл. Сложность алгоритма линейна по размеру графа. Визуализация делает наглядными и уменьшение степеней, и работу очереди, поэтому такие программы используют при изучении курсов алгоритмов.

1.2 Задачи практики

В рамках практики поставлены следующие задачи:

— задача 1: вводное задание — перевести на язык Java лабораторную работу, в которой использовалась структура данных на динамической памяти, и собрать программу в исполнимый jar-файл;

— задача 2: составить спецификацию визуализатора топологической сортировки и план итеративной разработки, согласовать их с руководителем;

— задача 3: в составе бригады из трёх студентов итеративно разработать приложение-визуализатор в соответствии со спецификацией и планом, сдавая промежуточные версии (прототип, альфа, бета, финальная).

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1 Вводное задание: перевод лабораторной работы на язык Java

Для вводного задания выбрана лабораторная работа №3 по дисциплине «Построение и анализ алгоритмов» — вычисление редакционного расстояния (расстояния Левенштейна) между двумя строками с использованием метода динамического программирования. Работа подходит под условие задания: в ней реализован классический алгоритм заполнения матрицы стоимостей размером $(n+1) \times (m+1)$, где каждая ячейка вычисляется на основе трёх операций — вставки, удаления и замены символа.

При переводе с Python на Java программа сохранена в структуре одного класса Main с методом solve(). Основные структуры данных — двумерные массивы `int[][] ss` для хранения стоимостей и `char[][] op` для хранения операций — выделяются в куче динамически с использованием оператора `new` в зависимости от длин входных строк. Для хранения последовательности операций используется `StringBuilder`, что обеспечивает эффективную конкатенацию при восстановлении предписания обратным ходом. Ввод данных осуществляется через `Scanner`, стоимости операций задаются пользователем, что позволяет гибко настраивать алгоритм. Пошаговый вывод процесса заполнения таблицы реализован с использованием форматированного вывода через `System.out.printf`, что обеспечивает наглядность работы алгоритма.

Пример выполнения. Вход: строки `S = "pedestal"`, `T = "stien"`, стоимости замены = 1, удаления = 1, вставки = 1. Выход программы — минимальное расстояние Левенштейна, равное 7, а также пошаговая таблица преобразований и предписание оптимальной последовательности операций.

2.2. Составление спецификации программы и плана разработки

Исходная спецификация и план разработки были составлены до начала разработки и согласованы с руководителем. Ниже приведены исходные требования, план с распределением ролей.

2.2.1 Исходные требования к программе

2.2.1.1 Требования к вводу исходных данных. Граф задаётся двумя способами: из текстового файла (строки вида «V номер x y» — вершина с координатами в условной системе 0..1000, «E из в» — ориентированное ребро; пустые строки и строки с «#» игнорируются) и непосредственно в приложении мышью: щелчок по свободному месту холста создаёт вершину, щелчки по двум вершинам создают ребро от первой ко второй, перетаскивание перемещает вершину, щелчок правой кнопкой удаляет элемент. Ограничения: до 50 вершин, до 300 рёбер; петли и повторные рёбра запрещены, встречные рёбра допускаются.

2.2.1.2 Требования к визуализации. Вершины подписаны номерами, направление рёбер показано стрелками. Рядом с необработанной вершиной подписана её текущая входящая степень. Вершины в очереди готовых выделяются жёлтой заливкой, обработанные — зелёной с номером позиции в порядке, текущая — красным жирным контуром; удалённые алгоритмом рёбра рисуются серым пунктиром. Выполнение пошаговое, шаги мелкие: одна вершина обрабатывается за два шага (взятие из очереди и удаление исходящих рёбер). Каждый шаг сопровождается текстовым пояснением в журнале; последнее пояснение дублируется в строке состояния. Панель справа показывает очередь готовых вершин и построенный порядок. Граф масштабируется под размер окна. Эскиз интерфейса приведён на рис. 1.

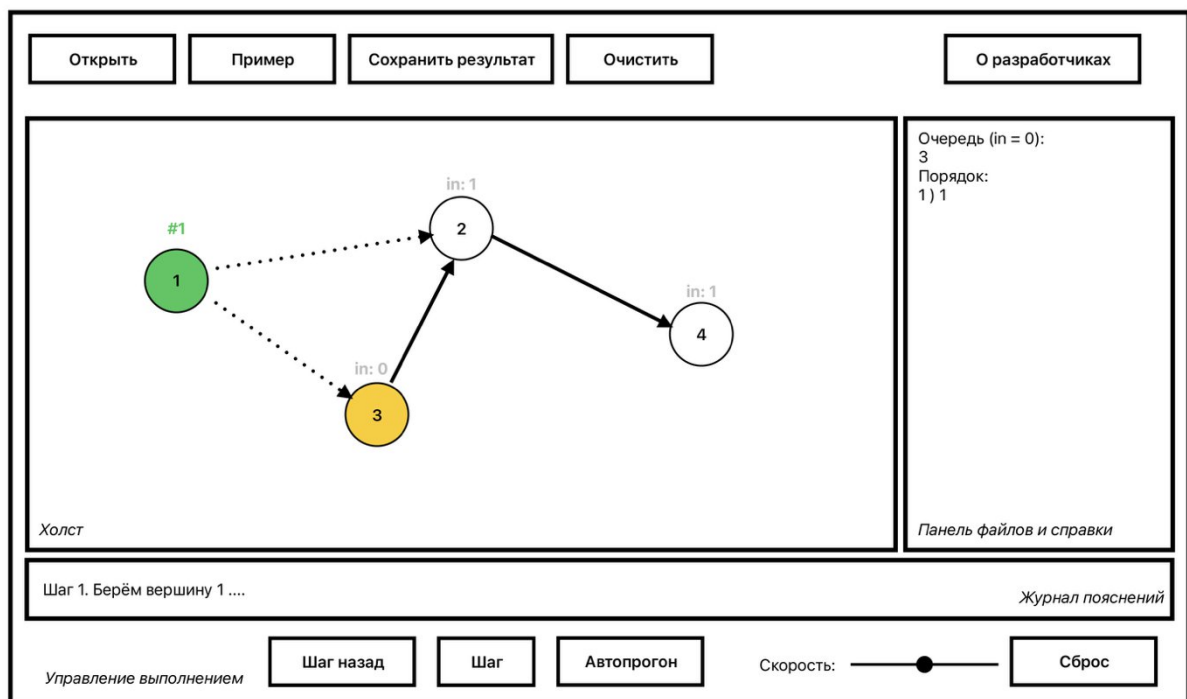


Рисунок 1 – Эскиз пользовательского интерфейса

2.2.1.3 Требования к управлению выполнением. Кнопки «Шаг» (один шаг вперёд), «Шаг назад» (откат на шаг), «Автопрогон» (автоматические шаги с интервалом, задаваемым ползунком 100..2000 мс; повторное нажатие — пауза), «Сброс» (возврат к началу). Во время выполнения редактирование графа блокируется до сброса.

2.2.1.4 Требования к выводу результатов и прочие требования. По завершении в журнал выводится топологический порядок; для графа с циклом — сообщение о невозможности сортировки с перечнем необработанных вершин и их подсветкой. Результат сохраняется в текстовый файл. Предусмотрены кнопка «О разработчиках» (ФИО, группа) и обработка ошибок: некорректный входной файл не приводит к аварийному завершению — выводится диалог с описанием ошибки и номером строки.

2.2.2 План разработки и распределение ролей в бригаде

План разработки по созвонам практики: к 10.07 — согласование спецификации и плана. К 13.07 — прототип: окно приложения по эскизу, холст с демонстрационным графом, состав кнопок. К 15.07 — альфа-версия: загрузка графа из файла, отрисовка, полный прогон алгоритма с выводом итога. К 17.07 — бета-версия: пошаговое выполнение с пояснениями и подсветкой, сохранение результата в файл. К 18.07 — финальная версия: шаг назад, автопрогон с задаваемым интервалом, редактирование графа мышью, обработка исключительных ситуаций, «О разработчиках», сборка исполнимого jar; вместе с финальной версией сдаётся отчёт.

Распределение ролей: Беспалов Александр Александрович — модель данных и алгоритм (граф, подсчёт степеней, пошаговый алгоритм Кана, форматы файлов, тесты модели). Мазеев Владислав Антонович — визуализация (холст, отрисовка графа со стрелками, подписи степеней и позиций, масштабирование, панель состояния). Серженко Дмитрий Кириллович — управление и интеграция (кнопки, таймер автопрогона, редактирование мышью, диалоги, обработка ошибок, сборка jar). Интеграция и взаимное ревью кода выполнялись совместно в общем репозитории. [2]

2.3 Командная итеративная разработка приложения в соответствии со спецификацией и планом

Разработка велась итеративно по плану из раздела 2.2.2: прототип, альфа-, бета- и финальная версии сдавались на созвонах в установленные сроки, существенных отклонений от плана не было. Приложение построено по схеме

«модель—представление—управление»: пакет `model` содержит граф и алгоритм и не зависит от интерфейса, пакет `ui` — окно и холст, пакет `io` — чтение и запись файлов. Такое разделение позволило разрабатывать части параллельно по ролям и проверять алгоритм автономными тестами без графического интерфейса.

2.3.1 Структуры данных

`Vertex` — вершина графа: номер и координаты в условной системе 0..1000. `Edge` — неизменяемое ориентированное ребро (запись `record` с полями `from` и `to`). `Graph` хранит списки вершин и рёбер и проверяет ограничения спецификации при добавлении: лимиты, запрет петель и повторных рёбер; в финальной версии добавлены операции удаления вершин и рёбер.

Входящие степени вершин хранятся в словаре `HashMap`: номера вершин задаются пользователем и могут идти не подряд. Очередь готовых вершин хранится отсортированным списком, из которого всегда берётся наименьший номер: результат детерминирован, что необходимо для шага назад и для тестов.

`TopoStepper` — пошаговая реализация алгоритма Кана в виде конечного автомата с фазами «старт — взятие вершины — удаление рёбер — итог». Объект хранит текущие степени, очередь готовых, построенный порядок и множество удалённых рёбер; рёбра из графа физически не удаляются, они лишь помечаются, чтобы граф оставался целым для отрисовки.

2.3.2 Основные методы

`GraphIO.load` — построчный разбор входного файла; каждая ошибка формата сопровождается номером строки. `GraphIO.saveResult` — запись результата в текстовый файл.

`TopoStepper.stepForward` — один шаг алгоритма, возвращающий текстовое пояснение для журнала. `TopoStepper.stepBack` — шаг назад: алгоритм сбрасывает

сывается и заново прокручивается на один шаг меньше; алгоритм детерминирован, поэтому состояние восстанавливается точно.

`GraphPanel.paintComponent` — отрисовка графа со стрелками, подписями степеней и позиций, раскраской вершин по состоянию и масштабированием координат под размер окна. Обработчики мыши `GraphPanel` создают, перемещают и удаляют элементы графа. `MainFrame` связывает кнопки, ползунок скорости и таймер автопрогона с моделью.

2.3.3 Тестирование

Тестирование выполнялось на трёх уровнях, описанных ниже.

2.3.3.1 План тестирования

План тестирования составлен к сдаче прототипа. Он включал модульные проверки модели (порядок на эталонном графе, обнаружение цикла, валидация ограничений, разбор корректных и некорректных файлов) и проверки пошагового выполнения: эквивалентность пошагового и полного прогона, точность отката, поведение на пустом графе. Интерфейс проверялся вручную по сценариям: загрузка из файла и кнопкой «Пример», редактирование мышью, автопрогон с изменением скорости, сохранение результата, реакция на некорректные файлы.

2.3.3.2 Тестирование графического интерфейса

Интерфейс проверен по сценариям плана. Внешний вид приложения после полного прогона алгоритма на демонстрационном графе показан на рис. 2: все вершины зелёные с позициями #1..#7, в журнале — итоговый порядок 1 3 2 5 4 7 6, панель справа содержит построенный порядок. Промежуточное состояние пошагового выполнения на графе с циклом показано на рис. 3: обработанные вершины зелёные, у необработанных подписаны текущие входящие степени, удалённые рёбра отображаются пунктиром, в панели справа видна очередь готовых. Проверены также диалог ошибки на некорректном файле (сообщение с номером строки), блокировка редактирования во время выполнения и диалог «О разработчиках».

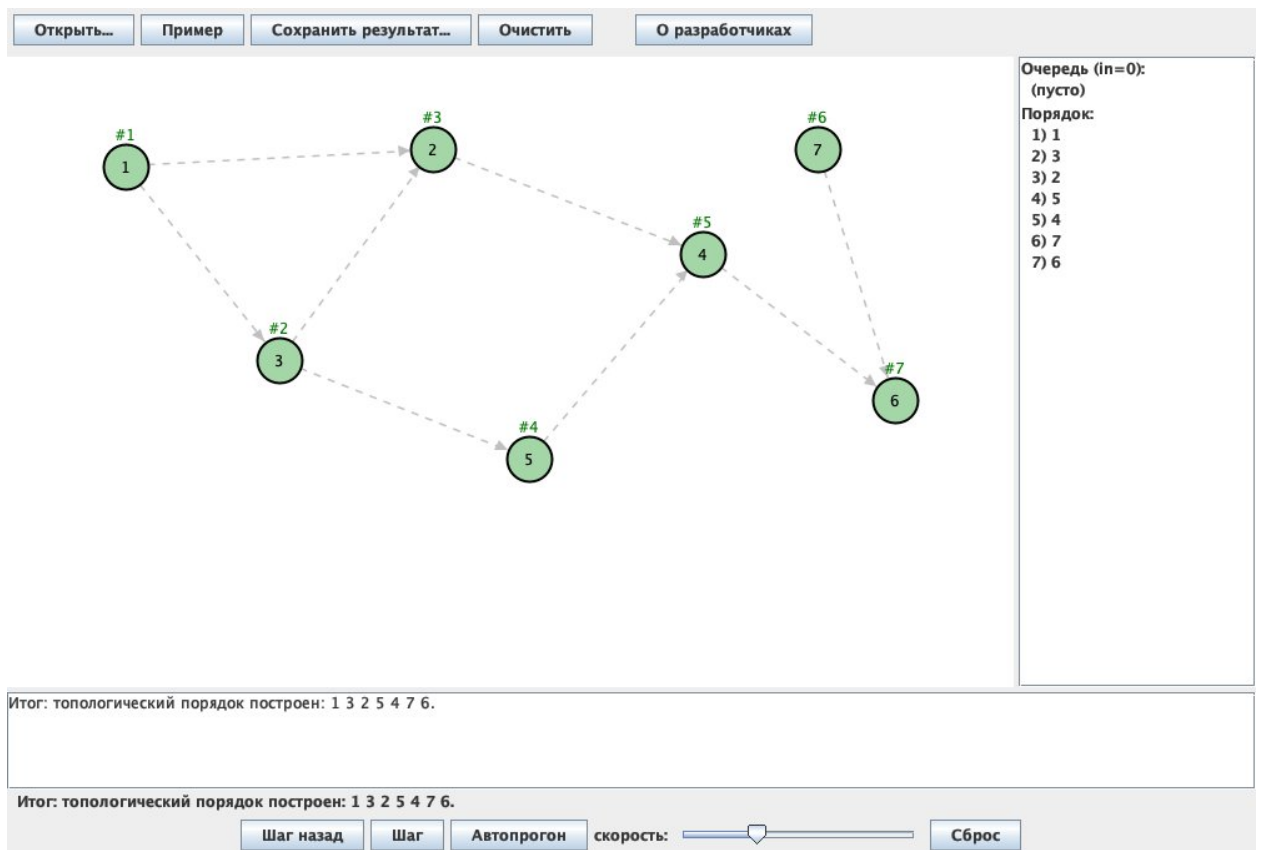


Рисунок 2 – Главное окно приложения после полного прогона алгоритма

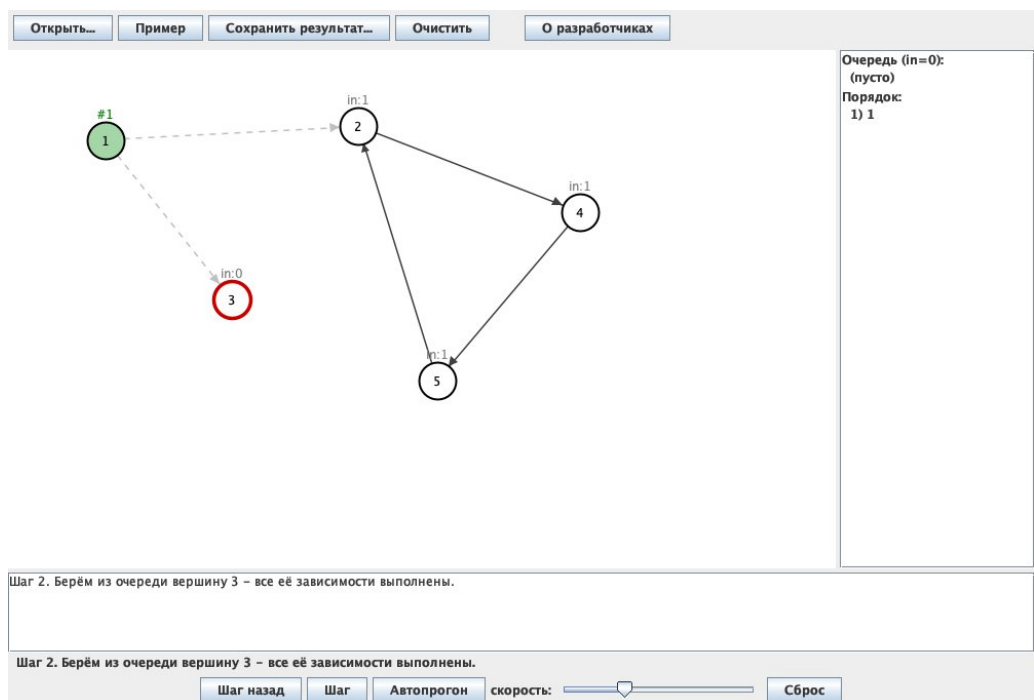


Рисунок 3 – Пошаговое выполнение на графе с циклом

2.3.3.3 Тестирование алгоритма

Модель проверена автономными тестами без графического интерфейса:
на эталонном графе из семи вершин и восьми рёбер алгоритм строит порядок

1 3 2 5 4 7 6, совпадающий с расчётом вручную; пошаговое выполнение даёт тот же результат, что и полный прогон; после отката и повторного прогона результат не меняется; цикл из трёх вершин обнаруживается с верным перечнем необработанных вершин; петли, повторные рёбра и ссылки на несуществующие вершины отклоняются с диагностикой. Все тесты пройдены.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Java 21 — язык разработки; использованы записи (record), перечисления, обобщённые коллекции, лямбда-выражения. Swing (входит в JDK) — библиотека графического интерфейса: окна, компоновка BorderLayout, отрисовка Graphics2D, диалоги JOptionPane и JFileChooser, таймер javax.swing.Timer для автопрогона [3]. Инструменты JDK: компилятор javac, архиватор jar.

Среда разработки — IntelliJ IDEA Community Edition. Совместная работа — система контроля версий Git и общий репозиторий бригады на GitHub; Сдачи и защиты — дистанционно.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Отзыв

о прохождении учебной практики

Серженко Дмитрий Кириллович, студент группы 4383, второго курса бакалавриата Санкт-Петербургского электротехнического университета «ЛЭТИ» им В.И. Ульянова, проходил учебную практику по теме «*Новые языки программирования*» «Топологическая сортировка» с 06.07.2026 по 18.07.2026.

В течение практики обучающийся Серженко Дмитрий Кириллович выполнял разработку модуля загрузки и сохранения графов, реализация обработки всех ошибок ввода(неверный формат, на несуществующие вершины, дубли, петли, превышение лимитов), подготовка тестовых сценариев и проверка работы программы на разных примерах.

В результате практики обучающийся успешно разработал рабочий модуль ввода-вывода, который корректно читает файлы, выдаёт понятные сообщения об ошибках с указанием строки, сохраняет результаты сортировки, и программа не падает при любых неправильных действиях пользователя. Также подготовлены и прогнаны все тесты, включая графы с циклами, пустые и битые файлы, все сценарии прошли успешно.

По результатам работы Серженко Дмитрий Кириллович учебную практику оцениваю на Отлично

Руководитель практики

Ассистент каф. МОЭВМ



Р.П. Шестопалов

15.07.2026

ЗАКЛЮЧЕНИЕ

В ходе учебной практики решены все поставленные задачи. Выполнено вводное задание: лабораторная работа со структурой данных на динамической памяти (расстояние Левенштейна) переведена с Python на Java и собрана в исполнимый jar-файл. Составлена и согласована спецификация визуализатора топологической сортировки с планом итеративной разработки и распределением ролей. Бригадой из трёх студентов разработано приложение с графическим интерфейсом: пошаговая визуализация алгоритма Кана с текстовыми пояснениями, шаг вперёд и назад, автопрогон с задаваемым интервалом, загрузка графа из файла и мышью, обнаружение циклов, сохранение результата, обработка некорректных входных данных. Приложение собрано в исполнимый jar-файл (Java 21). Практика позволила освоить язык Java, библиотеку Swing и технологию командной итеративной разработки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кормен, Т., Лейзерсон, Ч., Ривест, Р., Штайн, К. Алгоритмы: построение и анализ / Т. Кормен и др. — 3-е изд. — М.: Вильямс, 2013. — 1328 с.
2. Репозиторий мини-проекта бригады [Электронный ресурс]. URL: <https://github.com/yoonzky/TopologicalSortVisualization> (дата обращения: 17.07.2026).
3. Java Platform, Standard Edition 21 API Specification [Электронный ресурс]. URL: <https://docs.oracle.com/en/java/javase/21/docs/api/> (дата обращения: 17.07.2026).

ПРИЛОЖЕНИЕ А

Формат входного файла графа (текстовый, кодировка UTF-8). Каждая строка задаёт одну запись: «V номер x y» — вершина с координатами в условной системе 0..1000; «E из в» — ориентированное ребро от вершины «из» к вершине «в». Пустые строки и строки, начинающиеся с символа «#», игнорируются.

Пример файла (граф из семи вершин и восьми рёбер): V 1 100 150; V 2 420 120; V 3 260 480; V 4 700 300; V 5 520 650; V 6 900 550; V 7 820 120; E 1 2; E 1 3; E 3 2; E 2 4; E 3 5; E 5 4; E 4 6; E 7 6 (в файле каждая запись располагается на отдельной строке).

Результат работы приложения сохраняется в текстовый файл: топологический порядок вершин, а для графа с циклом — сообщение о невозможности сортировки и список необработанных вершин.